



LEARNER GUIDE

FOR

Data Mining and Machine Learning Fundamentals for Beginners

Course Code: TGS-2020503264

Conducted by Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 17.0 · 10 September 2026

Document Version Control Record

Version	Effective date	Summary of changes	Author
17.0	10 September 2026	Replaced legacy visual workflows with one Python and Streamlit app; expanded technical mechanisms, 15 self-contained labs, verification, troubleshooting, and assessment alignment.	Dr Alfred Ang

Course Information

Course	Data Mining and Machine Learning Fundamentals for Beginners
Course code	TGS-2020503264
Duration	2 days · 16 hours, plus 2 assessment hours
TSC	Data Analytics · MED-ACE-3018-1
Lab application	Run `streamlit run app.py`, then open http://localhost:8501

Learning Outcomes

- LO1: Apply data mining and machine learning principles to assess business insights.
- LO2: Integrate and aggregate information from multiple datasets to build data models.
- LO3: Apply predictive data modelling techniques to identify underlying trends in data.
- LO4: Apply machine learning classification techniques to gain new insights from data.
- LO5: Apply clustering techniques to discover patterns, visualise evidence, and make decisions.
- LO6: Develop prototype algorithms with dimension-reduction and feature-ranking techniques.
- LO7: Construct association rules to identify patterns across transactional datasets.

Central App Setup

1. Open Terminal in the repository root.
2. Create a virtual environment: `python3 -m venv .env`
3. Activate it on macOS/Linux: `source .env/bin/activate`
4. Install requirements: `python -m pip install -r requirements.txt`
5. Start the app: `streamlit run app.py`
6. Open the [Central Lab Portal](#), confirm the workflow and dataset, then open <http://localhost:8501> and upload the dataset from the current lab folder.

Topic 1 — Overview of Data Mining and Machine Learning

Alignment: LO1 · A8 · K4. From business question to monitored evidence.

CRISP-DM evidence loop

Mechanism. Business understanding → Data understanding → Preparation → Modelling → Evaluation → Deployment

Data/model contract. objective: measurable decision; unit_of_analysis: one row; target: observed outcome; acceptance: metric + threshold

Formula or decision rule. decision value = benefit of correct action – cost of errors

Worked example. \$80 benefit – \$18 expected error cost = \$62 net value

Evidence to read. expected decision value

Failure mode. Optimising a convenient model metric that does not change the business decision.

Control. Write the decision, owner, action, and cost matrix before modelling.

Verification. A reviewer can trace every model output to a decision and acceptance rule.

Machine-learning system boundary

Mechanism. Source system → Feature pipeline → Model → Decision rule → Human action → Outcome log

Data/model contract. inputs: versioned fields; model_artifact: fitted parameters; threshold: operating point; outcome: delayed ground truth

Formula or decision rule. prediction ≠ decision; decision = policy(prediction, context)

Worked example. 0.73 churn probability + threshold 0.65 → retention review

Evidence to read. decision trace coverage

Failure mode. Treating a prediction as an instruction and hiding ownership of the final action.

Control. Separate inference, policy, and human approval in the architecture.

Verification. Logs contain input version, score, threshold, action, and later outcome.

Supervised versus unsupervised evidence

Mechanism. Labelled rows → Loss function → Fitted predictor → Hold-out score

Data/model contract. supervised: X plus y; unsupervised: X only; classification: discrete y; regression: continuous y

Formula or decision rule. supervised minimises error to y; clustering minimises within-group distance

Worked example. churn label trains a classifier; spend patterns form clusters without labels

Evidence to read. external or internal validation

Failure mode. Calling clusters 'true customer types' without external validation.

Control. Use business review, stability tests, and later outcomes to validate segments.

Verification. The evaluation method matches the presence or absence of ground truth.

Train-evaluate-deploy lifecycle

Mechanism. Split → Fit → Validate → Freeze → Serve → Monitor

Data/model contract. train: parameter learning; validation: selection; test: final estimate; production: unseen stream

Formula or decision rule. generalisation gap = training score – test score

Worked example. 0.96 train – 0.81 test = 0.15 gap

Evidence to read. generalisation gap

Failure mode. Repeatedly tuning against the test set leaks test information into model selection.

Control. Lock a final test set and record experiment parameters separately.

Verification. Re-running the experiment with the same seed reproduces the split and score.

Business insight impact

Mechanism. Pattern → Hypothesis → Action → Experiment → Measured outcome

Data/model contract. segment: affected population; action: operational change; guardrail: harm limit; outcome: causal measure

Formula or decision rule. $\text{uplift} = \text{outcome_treatment} - \text{outcome_control}$

Worked example. 18.4% retention – 14.1% control = 4.3 pp uplift

Evidence to read. uplift and guardrails

Failure mode. Reporting correlation as causal impact.

Control. Use controlled trials or explicitly label observational evidence.

Verification. The insight statement includes scope, uncertainty, expected action, and limitation.

Topic 2 — Data Preparation

Alignment: LO2 · A1/A6 · K1/K7. Reliable tables before reliable models.

Schema and data types

Mechanism. Ingest → Profile → Coerce → Validate → Quarantine

Data/model contract. customer_id: string key; event_time: datetime; amount: float ≥ 0 ; target: explicit role

Formula or decision rule. $\text{validity rate} = \text{valid rows} / \text{received rows}$

Worked example. 9,720 valid / 10,000 received = 97.2%

Evidence to read. validity rate

Failure mode. Silent type coercion turns invalid values into missing values without an audit trail.

Control. Validate ranges and log rejected rows before conversion.

Verification. Row counts reconcile across received, accepted, and quarantined records.

Filtering and row selection

Mechanism. Predicate → Boolean mask → Subset → Row-count reconciliation

Data/model contract. predicate: amount > 0 ; scope: date window; null policy: explicit; index: preserved identifier

Formula or decision rule. $\text{selected} = \sum I(\text{condition is true})$

Worked example. 1,250 rows satisfy region='North' and amount >0

Evidence to read. selection rate

Failure mode. A filter unintentionally removes nulls because null comparisons evaluate false.

Control. Profile null counts before and after every predicate.

Verification. Selected + excluded + null-policy rows equals the source row count.

Join and concatenate

Mechanism. Key audit → Cardinality test → Join → Indicator review → Reconcile

Data/model contract. key: customer_id; left cardinality: many; right cardinality: one; join type: inner/left

Formula or decision rule. join expansion = output rows / left rows

Worked example. 168 output / 160 orders = 1.05× warns of duplicate customer keys

Evidence to read. match rate and expansion

Failure mode. Many-to-many keys duplicate facts and inflate revenue.

Control. Assert key uniqueness on the dimension table and inspect merge indicators.

Verification. Revenue before and after the intended join reconciles within the documented scope.

Missing-value treatment

Mechanism. Diagnose mechanism → Choose statistic → Fit on train → Transform → Flag imputed rows

Data/model contract. MCAR/MAR/MNAR hypothesis; numeric: median/mean; category: mode/Unknown; indicator: was_missing

Formula or decision rule. median = middle ordered value; robust to extreme values

Worked example. [20, 22, 24, 95] → median 23, mean 40.25

Evidence to read. distribution shift

Failure mode. Computing imputation values on the full dataset leaks test distribution.

Control. Fit imputers only on training data inside a pipeline.

Verification. No unintended nulls remain and train/test preprocessing is identical.

Scaling, sampling, and aggregation

Mechanism. Stratify → Fit scaler → Transform → Aggregate → Check leakage

Data/model contract. $z = (x - \mu) / \sigma$; min-max = $(x - \min) / (\max - \min)$; group key: declared; measure: count/sum/mean

Formula or decision rule. $z = (850 - 500) / 140 = 2.50$

Worked example. Revenue 850 is 2.5 standard deviations above training mean

Evidence to read. standardised range and group totals

Failure mode. Scaling before splitting lets test statistics influence training features.

Control. Split first; fit the scaler within the training pipeline.

Verification. Training features centre near zero and group totals reconcile to source totals.

Topic 3 — Regression

Alignment: LO3 · A2 · K2/K10. Predicting continuous outcomes with controlled error.

Linear model mechanism

Mechanism. Feature matrix $X \rightarrow$ Coefficients $\beta \rightarrow$ Intercept $\rightarrow$ Prediction $\hat{y} \rightarrow$ Residual e

Data/model contract. row: observation; column: feature; β : marginal slope; $\hat{y}$: numeric outcome

Formula or decision rule. $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$

Worked example. $\hat{y} = 120,000 + 5,200 \times 92 + 7,200 \times 10 = \$670,400$

Evidence to read. coefficient and residual

Failure mode. Interpreting a coefficient causally when correlated variables or selection bias are present.

Control. Use coefficients as conditional associations unless the design supports causality.

Verification. Predictions, coefficients, and residuals can be reproduced from the frozen pipeline.

Residual diagnostics

Mechanism. Predict $\rightarrow$ Compute residual $\rightarrow$ Plot against fitted $\rightarrow$ Check pattern $\rightarrow$ Revise specification

Data/model contract. residual $e = y - \hat{y}$; zero-centred cloud; constant variance; independent errors

Formula or decision rule. $SSE = \sum (y_i - \hat{y}_i)^2$

Worked example. errors [10, -5, 15] $\rightarrow SSE = 100 + 25 + 225 = 350$

Evidence to read. residual pattern

Failure mode. A funnel shape means error variance grows with the prediction.

Control. Transform the target, add relevant features, or use robust intervals.

Verification. Residuals show no systematic curve, drift, or group-specific bias.

MAE, MSE, RMSE, and R^2

Mechanism. Error vector $\rightarrow$ Absolute/squared loss $\rightarrow$ Average $\rightarrow$ Scale recovery $\rightarrow$ Baseline comparison

Data/model contract. MAE: linear penalty; MSE: squared penalty; RMSE: target units; R^2 : versus mean baseline

Formula or decision rule. $R^2 = 1 - SSE/SST$

Worked example. SSE 1,800; SST 9,000 $\rightarrow R^2 = 0.80$

Evidence to read. hold-out MAE/RMSE/ R^2

Failure mode. High R^2 can coexist with large business-critical errors in a subgroup.

Control. Report absolute error and segment diagnostics alongside R^2 .

Verification. Metrics are computed on unseen rows and compared with a simple baseline.

Underfitting, overfitting, and validation

Mechanism. Candidate complexity → Training error → Validation error → Select → Final test

Data/model contract. underfit: both errors high; fit: low validation error; overfit: widening gap; test: used once

Formula or decision rule. gap = train RMSE – test RMSE

Worked example. train 12k vs test 47k → –35k gap indicates overfit

Evidence to read. validation curve

Failure mode. Selecting the deepest model because it minimises training error.

Control. Choose complexity at the validation minimum and retain a final test set.

Verification. Learning curves show whether more data or a simpler model is likely to help.

Ridge and Lasso regularisation

Mechanism. Scale features → Add penalty → Optimise → Shrink coefficients → Validate α

Data/model contract. Ridge: L2 penalty; Lasso: L1 penalty; α : penalty strength; intercept: usually unpenalised

Formula or decision rule. Ridge: minimise $SSE + \alpha \sum \beta^2$; Lasso: $SSE + \alpha \sum |\beta|$

Worked example. $\alpha=1$ shrinks unstable coefficients while retaining all Ridge features

Evidence to read. cross-validated error versus α

Failure mode. Comparing penalty strengths on unscaled features makes the penalty uneven.

Control. Standardise within the pipeline before Ridge or Lasso.

Verification. Selected α improves or preserves validation error without collapsing useful signal.

Topic 4 — Classification

Alignment: LO4 · A7 · K6. Separating classes and choosing an operating threshold.

Decision tree and ensemble path

Mechanism. Root split → Impurity reduction → Child nodes → Leaf probability → Threshold

Data/model contract. feature threshold; Gini/entropy; max_depth; min_samples_leaf

Formula or decision rule. $Gini = 1 - \sum p_k^2$

Worked example. $p(\text{churn})=0.7 \rightarrow Gini=1-0.7^2-0.3^2=0.42$

Evidence to read. depth, leaf support, validation score

Failure mode. Deep leaves memorise rare combinations and become unstable.

Control. Limit depth/leaf size and compare with bagged Random Forest estimates.

Verification. Leaf counts, feature importance, and hold-out scores remain plausible.

Logistic, KNN, SVM, and Naive Bayes

Mechanism. Transform features → Fit decision surface → Score class → Apply threshold

Data/model contract. logistic: log-odds; KNN: neighbourhood vote; SVM: margin/kernel; NB: conditional likelihood

Formula or decision rule. $\text{logit}(p) = \beta_0 + \beta x$; $p = 1/(1 + \exp(-z))$

Worked example. $z = 1.386 \rightarrow p \approx 0.80$

Evidence to read. probability calibration and discrimination

Failure mode. KNN and SVM are distorted when features have incompatible scales.

Control. Use a shared preprocessing pipeline and compare models under identical folds.

Verification. Each classifier is evaluated on the same rows, labels, and metrics.

Stratified K-fold cross-validation

Mechanism. Shuffle → Preserve class ratio → Fit K-1 folds → Score held-out fold → Aggregate

Data/model contract. k: number of folds; stratify: class balance; seed: reproducibility; mean ± standard deviation

Formula or decision rule. $\text{CV mean} = \sum \text{score}_i / k$

Worked example. $[\text{.84}, \text{.87}, \text{.85}, \text{.86}, \text{.88}] \rightarrow \text{mean } .86$

Evidence to read. mean and fold variability

Failure mode. Preprocessing outside the cross-validation loop leaks fold information.

Control. Put imputation, encoding, scaling, and model in one pipeline.

Verification. Every row is held out once and class proportions are stable across folds.

Confusion matrix and class metrics

Mechanism. Compare labels → Count TP/TN/FP/FN → Compute metrics → Attach costs

Data/model contract. precision = $TP/(TP+FP)$; recall = $TP/(TP+FN)$; specificity = $TN/(TN+FP)$; F1 harmonic mean

Formula or decision rule. $F1 = 2PR/(P+R)$

Worked example. $P = .75, R = .60 \rightarrow F1 = .667$

Evidence to read. precision, recall, F1, specificity

Failure mode. Accuracy hides failure when the positive class is rare.

Control. Report the full matrix and select metrics based on error cost.

Verification. Matrix cell counts sum to the evaluated sample size.

ROC, AUC, and threshold policy

Mechanism. Probability score → Candidate threshold → TPR/FPR → ROC curve → Operating point

Data/model contract. TPR=recall; FPR=FP/(FP+TN); AUC: ranking probability; threshold: policy choice

Formula or decision rule. expected cost = $FP \times C_{FP} + FN \times C_{FN}$

Worked example. $15 FP \times \$20 + 30 FN \times \$120 = \$3,900$

Evidence to read. AUC plus operating cost

Failure mode. Choosing 0.5 by convention when false negatives are much costlier.

Control. Select a threshold from business costs and capacity constraints.

Verification. The chosen point states threshold, TPR, FPR, volume, and expected cost.

Topic 5 — Clustering

Alignment: LO5 · A5/A9 · K3/K5. Discovering structure without labels.

Distance and scaling

Mechanism. Select features → Impute → Scale → Compute pairwise distance

Data/model contract. Euclidean: continuous; Manhattan: absolute steps; cosine: direction; categorical: requires encoding

Formula or decision rule. $d(x,y) = \sqrt{\sum (x_i - y_i)^2}$

Worked example. (2,3) to (5,7) → $\sqrt{(9+16)}=5$

Evidence to read. distance distribution

Failure mode. A high-magnitude spend field dominates an unscaled visit-frequency field.

Control. Standardise numeric variables and justify the distance metric.

Verification. Nearest neighbours are sensible under domain review and robust to scaling choices.

K-means assignment/update loop

Mechanism. Initial centroids → Assign nearest → Recompute means → Repeat → Converge

Data/model contract. k: requested clusters; centroid: feature mean; inertia: within-cluster SSE; n_init: multiple starts

Formula or decision rule. $inertia = \sum ||x_i - \mu_{cluster(i)}||^2$

Worked example. three compact groups reduce inertia from 940 to 218

Evidence to read. inertia and stability

Failure mode. One random start reaches a poor local minimum.

Control. Use multiple initialisations and compare label stability.

Verification. Centroids, inertia, and labels reproduce under the recorded seed.

Silhouette analysis

Mechanism. Within distance $a \rightarrow$ Nearest-cluster distance $b \rightarrow$ Score each point $\rightarrow$ Average by k

Data/model contract. range: -1 to 1 ; near 1 : separated; near 0 : boundary; negative: likely misassigned

Formula or decision rule. $s=(b-a)/\max(a,b)$

Worked example. $a=2, b=5 \rightarrow s=0.60$

Evidence to read. mean and per-cluster silhouette

Failure mode. Selecting the highest average while ignoring a tiny or unstable cluster.

Control. Review the distribution, cluster sizes, and business usefulness.

Verification. The chosen k has acceptable separation and interpretable cluster sizes.

Hierarchical clustering and dendrogram

Mechanism. Singleton clusters $\rightarrow$ Pairwise distances $\rightarrow$ Ward merge $\rightarrow$ Update tree $\rightarrow$ Cut height

Data/model contract. linkage: Ward; merge distance; dendrogram height; cut: cluster count

Formula or decision rule. Ward merges the pair with the smallest increase in within-cluster variance

Worked example. large jump from 6.2 to 14.8 suggests a cut before the jump

Evidence to read. merge-distance jump

Failure mode. Treating horizontal spacing in a dendrogram as a numeric distance.

Control. Read merge distance from the vertical axis and document the cut height.

Verification. The cut produces the stated cluster count and matching scatter colours.

Cluster interpretation and limitations

Mechanism. Profile clusters $\rightarrow$ Compare distributions $\rightarrow$ Name cautiously $\rightarrow$ Test stability $\rightarrow$ Decide action

Data/model contract. size: n and $\%$; centroid: scaled profile; stability: repeated samples; action: testable intervention

Formula or decision rule. standardised lift = cluster mean $-$ overall mean

Worked example. cluster A spend $z=+1.2$ and visits $z=+0.8$

Evidence to read. profile lift and stability

Failure mode. Narrative labels exaggerate weak numerical differences.

Control. Publish measured traits, uncertainty, and exclusions before naming segments.

Verification. Cluster actions are tested and do not encode prohibited discrimination.

Topic 6 — Dimension Reduction

Alignment: LO6 · A4 · K8. Compressing variation while retaining evidence.

Curse of dimensionality

Mechanism. Add features → Distances concentrate → Neighbourhoods empty → Variance rises

Data/model contract. p: feature count; n: sample count; sparsity: volume effect; redundancy: correlated inputs

Formula or decision rule. required coverage grows exponentially with p

Worked example. 10 bins per feature → 10^6 cells for 6 features

Evidence to read. distance contrast

Failure mode. Adding weak features makes neighbours less distinguishable.

Control. Remove leakage/redundancy and validate feature additions.

Verification. Validation improves or remains stable when a feature is retained.

Standardisation and covariance

Mechanism. Centre → Scale → Covariance matrix → Eigen decomposition

Data/model contract. mean vector μ ; standard deviation σ ; covariance Σ ; symmetric matrix

Formula or decision rule. $\text{cov}(x,y) = \Sigma(x-\bar{x})(y-\bar{y})/(n-1)$

Worked example. positive covariance means the two centred features move together

Evidence to read. covariance structure

Failure mode. PCA on raw units lets high-variance currency fields dominate ratios.

Control. Standardise features when units and scales differ.

Verification. Each scaled feature has near-zero mean and unit variance on training data.

Principal components

Mechanism. Compute eigenvectors → Order eigenvalues → Project rows → Retain components

Data/model contract. loading: feature weight; score: projected row; eigenvalue: variance; orthogonal axes

Formula or decision rule. $Z = XW$

Worked example. two correlated measures rotate into PC1 along shared variation

Evidence to read. explained variance ratio

Failure mode. Interpreting component sign as inherently positive or negative; signs may flip.

Control. Interpret relative loading magnitude and relationships, not arbitrary sign alone.

Verification. Projected scores reproduce from saved scaler and PCA objects.

Retained variance and loadings

Mechanism. Fit full PCA → Cumulative sum → Find threshold crossing → Refit n components → Inspect loadings

Data/model contract. variance target: 0.70; n: first crossing; loading magnitude; reconstruction loss

Formula or decision rule. $n = \min\{k: \sum_{i \leq k} \text{variance}_i \geq .70\}$

Worked example. 46% + 27% = 73% → retain 2 components

Evidence to read. cumulative explained variance

Failure mode. Choosing 70% mechanically when the discarded component contains rare critical behaviour.

Control. Review reconstruction errors and downstream performance.

Verification. The selected n is the first threshold crossing and is recorded with the data version.

Feature ranking

Mechanism. Fit baseline → Permute one feature → Rescore → Measure drop → Repeat

Data/model contract. importance: score decrease; repeats: uncertainty; correlation: shared credit; target: defined

Formula or decision rule. $\text{importance}_j = \text{score}(X) - \text{score}(\text{permute } X_j)$

Worked example. .91 baseline – .78 permuted = .13 importance

Evidence to read. permutation importance

Failure mode. Correlated variables split or mask importance and ranking is mistaken for causality.

Control. Group correlated features and validate rankings on hold-out data.

Verification. Importance is positive, repeatable, and connected to an auditable model score.

Topic 7 — Association Analysis

Alignment: LO7 · A3 · K9. From baskets to defensible co-occurrence rules.

Transaction-to-basket encoding

Mechanism. Transaction rows → Group by ID → One-hot items → Boolean basket

Data/model contract. transaction_id: grouping key; item: canonical label; cell: present/absent; duplicates: collapse

Formula or decision rule. $\text{basket}[t,i] = 1$ if item i occurs in transaction t

Worked example. T001 milk+diapers+beer → three True cells

Evidence to read. basket density

Failure mode. Treating purchase quantity as repeated independent transactions.

Control. Define the basket unit and collapse duplicate item lines per transaction.

Verification. One encoded row exists per transaction and item counts reconcile.

Support

Mechanism. Count baskets containing itemset → Divide by all baskets → Apply threshold

Data/model contract. support(A); support($A \cap B$); minimum support; denominator: all transactions

Formula or decision rule. $\text{support}(A) = \text{count}(A) / N$

Worked example. beer appears in 8 of 15 baskets → support=.533

Evidence to read. support

Failure mode. A high minimum support removes valuable niche patterns.

Control. Tune support to transaction volume and decision value.

Verification. Manual counts for selected itemsets match the mined result.

Confidence and lift

Mechanism. Select antecedent → Count joint occurrence → Condition → Normalise by consequent prevalence

Data/model contract. $\text{confidence}(A \rightarrow B) = \text{support}(A \cap B) / \text{support}(A)$; $\text{lift} = \text{confidence} / \text{support}(B)$; $\text{lift} > 1$: positive association; direction matters

Formula or decision rule. $\text{lift}(A \rightarrow B) = \text{support}(A \cap B) / (\text{support}(A) \text{support}(B))$

Worked example. $.40 / (.50 \times .50) = 1.60$

Evidence to read. confidence and lift

Failure mode. High confidence for a universally popular item is mistaken for a useful rule.

Control. Require lift above 1 and assess absolute support.

Verification. The rule includes support, confidence, lift, and basket counts.

Apriori candidate pruning

Mechanism. Frequent 1-itemsets → Join candidates → Prune infrequent subsets → Count support → Repeat

Data/model contract. downward closure; candidate level k; minimum support; frequent itemset

Formula or decision rule. if itemset S is infrequent, every superset of S is infrequent

Worked example. {milk,diapers} below threshold → prune all triples containing both

Evidence to read. candidate count by level

Failure mode. A low threshold creates a combinatorial explosion of candidates.

Control. Start with a defensible support floor and cap itemset length.

Verification. Candidate counts shrink after pruning and results reproduce at the same threshold.

Rule filtering and business action

Mechanism. Generate rules → Apply thresholds → Remove redundancy → Review plausibility → Test offer

Data/model contract. antecedent: trigger basket; consequent: suggested item; leverage: excess joint rate; action: controlled test

Formula or decision rule. $\text{leverage} = \text{support}(A \cap B) - \text{support}(A)\text{support}(B)$

Worked example. $.40 - (.50 \times .50) = .15$

Evidence to read. incremental basket outcome

Failure mode. Deploying a bundle solely because lift is high on a tiny sample.

Control. Require minimum counts and validate with a controlled promotion.

Verification. The action has an owner, audience, guardrail, and measured outcome.

Detailed Hands-on Labs

Lab 01 — Data audit and business question

Alignment: LO1 · K1 · A8

Central-app workflow: Data audit & preparation

Mock data: customer_quality.csv

Identify data types, missingness, duplicates, outliers, and the decision that the evidence can support.

- Use Python 3.11 or later.
 - Work from the repository root so every lab uses the same central application.
 - Keep the supplied mock data unchanged; export results under a new filename.
7. Open Terminal and change to the repository root (the folder containing app.py).
 8. Create and activate a virtual environment:
 9. Install the shared dependencies once:
 10. Start the central application:
 11. Open <http://localhost:8501>, choose Data audit & preparation, and upload labs/lab-01-audit-business-data/customer_quality.csv.
 12. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.
 13. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
 14. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The profile reports 120 rows and at least three columns with missing values.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

15. Return to Terminal and press Control-C to stop Streamlit.

16. Keep only your exported evidence files; do not overwrite the supplied CSV.

17. Run deactivate when you have finished the lab session.

Lab 02 — Clean, impute, and export

Alignment: LO2 · K1 · A1

Central-app workflow: Data audit & preparation

Mock data: `customer_quality.csv`

Apply median/mode imputation and export a reproducible cleaned dataset.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

18. Open Terminal and change to the repository root (the folder containing `app.py`).

19. Create and activate a virtual environment:

20. Install the shared dependencies once:

21. Start the central application:

22. Open <http://localhost:8501>, choose Data audit & preparation, and upload `labs/lab-02-clean-impute-export/customer_quality.csv`.

23. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.

24. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.

25. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The downloaded file contains no blank cells in the profiled columns.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

26. Return to Terminal and press Control-C to stop Streamlit.

27. Keep only your exported evidence files; do not overwrite the supplied CSV.
28. Run deactivate when you have finished the lab session.

Lab 03 — Filter, join, and aggregate

Alignment: LO2 · K1 · A1/A6

Central-app workflow: Join & aggregate

Mock data: orders.csv

Join order and customer tables, retain matched records, and aggregate revenue by segment.

- Use Python 3.11 or later.
 - Work from the repository root so every lab uses the same central application.
 - Keep the supplied mock data unchanged; export results under a new filename.
29. Open Terminal and change to the repository root (the folder containing app.py).
 30. Create and activate a virtual environment:
 31. Install the shared dependencies once:
 32. Start the central application:
 33. Open <http://localhost:8501>, choose Join & aggregate, and upload labs/lab-03-filter-join-aggregate/orders.csv.
 34. In Join & aggregate, upload customers.csv as the second table, choose customer_id, and select an inner join.
 35. Group by segment and measure revenue.
 36. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
 37. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The inner join contains 160 orders; the Enterprise segment has the highest total revenue.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

38. Return to Terminal and press Control-C to stop Streamlit.
39. Keep only your exported evidence files; do not overwrite the supplied CSV.
40. Run deactivate when you have finished the lab session.

Lab 04 — Scale features before modelling

Alignment: LO3 · K2/K10 · A2

Central-app workflow: Regression

Mock data: housing_scale.csv

Compare a linear model with and without a high-magnitude feature and explain why scaling matters for regularised models.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

41. Open Terminal and change to the repository root (the folder containing app.py).

42. Create and activate a virtual environment:

43. Install the shared dependencies once:

44. Start the central application:

45. Open <http://localhost:8501>, choose Regression, and upload
labs/lab-04-scale-and-compare/housing_scale.csv.

46. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.

47. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.

48. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

Ridge produces finite MAE, RMSE and R^2 metrics and the actual-versus-predicted points follow the diagonal.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

49. Return to Terminal and press Control-C to stop Streamlit.

50. Keep only your exported evidence files; do not overwrite the supplied CSV.

51. Run deactivate when you have finished the lab session.

Lab 05 — Build a regression baseline

Alignment: LO3 · K2/K5 · A2

Central-app workflow: Regression

Mock data: housing_regression.csv

Train a linear regression model and interpret residual error using MAE, RMSE, and R^2 .

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

52. Open Terminal and change to the repository root (the folder containing app.py).

53. Create and activate a virtual environment:

54. Install the shared dependencies once:

55. Start the central application:
56. Open `http://localhost:8501`, choose Regression, and upload `labs/lab-05-regression-baseline/housing_regression.csv`.
57. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.
58. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
59. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

R^2 is above 0.75 on the supplied deterministic dataset.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh `http://localhost:8501`.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

60. Return to Terminal and press Control-C to stop Streamlit.
61. Keep only your exported evidence files; do not overwrite the supplied CSV.
62. Run deactivate when you have finished the lab session.

Lab 06 — Control overfitting with Ridge and Lasso

Alignment: LO3 · K2/K10 · A2

Central-app workflow: Regression

Mock data: `housing_regression.csv`

Compare Linear, Ridge, and Lasso models while varying alpha and identify the bias-variance trade-off.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

63. Open Terminal and change to the repository root (the folder containing `app.py`).
64. Create and activate a virtual environment:
65. Install the shared dependencies once:
66. Start the central application:
67. Open `http://localhost:8501`, choose Regression, and upload `labs/lab-06-regularization-overfit/housing_regression.csv`.
68. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.
69. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
70. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

A moderate alpha keeps test error stable; an excessively large alpha reduces R^2 .

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

71. Return to Terminal and press Control-C to stop Streamlit.

72. Keep only your exported evidence files; do not overwrite the supplied CSV.

73. Run deactivate when you have finished the lab session.

Lab 07 — Compare classification algorithms

Alignment: LO4 · K6 · A7

Central-app workflow: Classification

Mock data: `breast_cancer.csv`

Compare Logistic Regression, Decision Tree, Random Forest, KNN, and SVM on the same target and features.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

74. Open Terminal and change to the repository root (the folder containing `app.py`).

75. Create and activate a virtual environment:

76. Install the shared dependencies once:

77. Start the central application:

78. Open <http://localhost:8501>, choose Classification, and upload `labs/lab-07-classification-baselines/breast_cancer.csv`.

79. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.

80. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.

81. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

At least one supplied classifier achieves accuracy above 0.90.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

82. Return to Terminal and press Control-C to stop Streamlit.
83. Keep only your exported evidence files; do not overwrite the supplied CSV.
84. Run deactivate when you have finished the lab session.

Lab 08 — Use stratified K-fold validation

Alignment: LO4 · K6 · A7

Central-app workflow: Cross-validation

Mock data: breast_cancer.csv

Estimate generalisation performance across five stratified folds instead of trusting a single split.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

85. Open Terminal and change to the repository root (the folder containing app.py).

86. Create and activate a virtual environment:

87. Install the shared dependencies once:

88. Start the central application:

89. Open <http://localhost:8501>, choose Cross-validation, and upload
labs/lab-08-kfold-validation/breast_cancer.csv.

90. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.

91. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.

92. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

Five accuracy bars appear and the reported mean lies between 0 and 1.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

93. Return to Terminal and press Control-C to stop Streamlit.

94. Keep only your exported evidence files; do not overwrite the supplied CSV.

95. Run deactivate when you have finished the lab session.

Lab 09 — Interpret confusion metrics and error costs

Alignment: LO4 · K6/K7 · A7/A8

Central-app workflow: Classification

Mock data: churn.csv

Train a churn classifier, inspect the confusion matrix, and compare precision, recall, F1, and ROC AUC.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

96. Open Terminal and change to the repository root (the folder containing app.py).

97. Create and activate a virtual environment:

98. Install the shared dependencies once:

99. Start the central application:

100. Open <http://localhost:8501>, choose Classification, and upload labs/lab-09-confusion-roc-costs/churn.csv.

101. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.

102. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.

103. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The confusion-matrix total equals the test-set size and all reported metrics are within [0,1].

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

104. Return to Terminal and press Control-C to stop Streamlit.

105. Keep only your exported evidence files; do not overwrite the supplied CSV.

106. Run deactivate when you have finished the lab session.

Lab 10 — Select K with silhouette evidence

Alignment: LO5 · K3/K5 · A5

Central-app workflow: Clustering

Mock data: customer_clusters.csv

Scan candidate K values, choose a defensible cluster count, and label customer segments.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

107. Open Terminal and change to the repository root (the folder containing app.py).

108. Create and activate a virtual environment:

109. Install the shared dependencies once:

110. Start the central application:

111. Open `http://localhost:8501`, choose Clustering, and upload `labs/lab-10-kmeans-silhouette/customer_clusters.csv`.
112. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.
113. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
114. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The strongest silhouette score occurs near $K=3$ and the scatter plot separates three groups.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh `http://localhost:8501`.
 - A column is missing: verify that you uploaded this lab's file, not a file from another lab.
 - A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
 - Metrics differ slightly: use the default random state and unchanged mock data.
 - Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.
115. Return to Terminal and press Control-C to stop Streamlit.
 116. Keep only your exported evidence files; do not overwrite the supplied CSV.
 117. Run deactivate when you have finished the lab session.

Lab 11 — Cut a hierarchical dendrogram

Alignment: LO5 · K3/K5 · A5/A6

Central-app workflow: Hierarchical clustering

Mock data: `patient_profiles.csv`

Use Ward linkage to inspect merge distances, select two clusters, and visualise the cut.

- Use Python 3.11 or later.
 - Work from the repository root so every lab uses the same central application.
 - Keep the supplied mock data unchanged; export results under a new filename.
118. Open Terminal and change to the repository root (the folder containing `app.py`).
 119. Create and activate a virtual environment:
 120. Install the shared dependencies once:
 121. Start the central application:
 122. Open `http://localhost:8501`, choose Hierarchical clustering, and upload `labs/lab-11-hierarchical-dendrogram/patient_profiles.csv`.
 123. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.
 124. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
 125. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

Two colours appear in the cluster plot and a dendrogram is rendered.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

126. Return to Terminal and press Control-C to stop Streamlit.

127. Keep only your exported evidence files; do not overwrite the supplied CSV.

128. Run deactivate when you have finished the lab session.

Lab 12 — Retain variance with PCA

Alignment: LO6 · K8 · A4/A5

Central-app workflow: PCA & feature ranking

Mock data: `patient_profiles.csv`

Standardise numeric features, retain at least 70% cumulative variance, and inspect component loadings.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

129. Open Terminal and change to the repository root (the folder containing `app.py`).

130. Create and activate a virtual environment:

131. Install the shared dependencies once:

132. Start the central application:

133. Open <http://localhost:8501>, choose PCA & feature ranking, and upload `labs/lab-12-pca-variance/patient_profiles.csv`.

134. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.

135. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.

136. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The chosen component count is the first point where cumulative variance reaches at least 0.70.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

137. Return to Terminal and press Control-C to stop Streamlit.
138. Keep only your exported evidence files; do not overwrite the supplied CSV.
139. Run deactivate when you have finished the lab session.

Lab 13 — Rank predictive features

Alignment: LO6 · K8 · A4

Central-app workflow: PCA & feature ranking

Mock data: breast_cancer.csv

Use permutation importance to rank which measurements most affect classification accuracy.

- Use Python 3.11 or later.
- Work from the repository root so every lab uses the same central application.
- Keep the supplied mock data unchanged; export results under a new filename.

140. Open Terminal and change to the repository root (the folder containing app.py).
141. Create and activate a virtual environment:
142. Install the shared dependencies once:
143. Start the central application:
144. Open <http://localhost:8501>, choose PCA & feature ranking, and upload labs/lab-13-feature-ranking/breast_cancer.csv.
145. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.
146. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
147. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The ranking table is ordered from the largest to the smallest mean importance.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
- A column is missing: verify that you uploaded this lab's file, not a file from another lab.
- A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
- Metrics differ slightly: use the default random state and unchanged mock data.
- Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.

148. Return to Terminal and press Control-C to stop Streamlit.
149. Keep only your exported evidence files; do not overwrite the supplied CSV.
150. Run deactivate when you have finished the lab session.

Lab 14 — Mine market-basket rules

Alignment: LO7 · K9 · A3/A4

Central-app workflow: Association rules

Mock data: market_basket.csv

Generate frequent itemsets and association rules; interpret support, confidence, and lift.

- Use Python 3.11 or later.
 - Work from the repository root so every lab uses the same central application.
 - Keep the supplied mock data unchanged; export results under a new filename.
151. Open Terminal and change to the repository root (the folder containing app.py).
 152. Create and activate a virtual environment:
 153. Install the shared dependencies once:
 154. Start the central application:
 155. Open <http://localhost:8501>, choose Association rules, and upload labs/lab-14-association-rules/market_basket.csv.
 156. Select transaction_id and item; begin with support 0.20 and confidence 0.60.
 157. Filter the rule table for the antecedent containing milk and diapers.
 158. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
 159. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The milk-and-diapers antecedent produces beer among its strong consequents at the supplied thresholds.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
 - A column is missing: verify that you uploaded this lab's file, not a file from another lab.
 - A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
 - Metrics differ slightly: use the default random state and unchanged mock data.
 - Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.
160. Return to Terminal and press Control-C to stop Streamlit.
 161. Keep only your exported evidence files; do not overwrite the supplied CSV.
 162. Run deactivate when you have finished the lab session.

Lab 15 — Detect and explain anomalies

Alignment: LO1 · K1/K5 · A8

Central-app workflow: Anomaly detection

Mock data: sensor_readings.csv

Compare Z-score and IQR flags and explain whether each extreme value warrants business action.

- Use Python 3.11 or later.
 - Work from the repository root so every lab uses the same central application.
 - Keep the supplied mock data unchanged; export results under a new filename.
163. Open Terminal and change to the repository root (the folder containing app.py).
 164. Create and activate a virtual environment:
 165. Install the shared dependencies once:
 166. Start the central application:

167. Open <http://localhost:8501>, choose Anomaly detection, and upload `labs/lab-15-anomaly-evidence/sensor_readings.csv`.
168. Adjust the available model or threshold control and compare the evidence rather than judging from one metric.
169. Record the chosen inputs, parameter values, output metrics, and one screenshot of the visual evidence.
170. Export the result when the workflow provides a download button, then state one business interpretation and one limitation.

The injected spikes at rows 41, 88, and 121 are flagged with a suitable threshold.

Your evidence is complete when it includes the uploaded filename, selected fields, parameters, result table or chart, and a plain-language interpretation.

- The page is blank: confirm Streamlit is running and refresh <http://localhost:8501>.
 - A column is missing: verify that you uploaded this lab's file, not a file from another lab.
 - A model fails: remove identifier/text fields from numeric-only feature selections and confirm the target is not also selected as a feature.
 - Metrics differ slightly: use the default random state and unchanged mock data.
 - Port 8501 is busy: run `streamlit run app.py --server.port 8502` and open the printed URL.
171. Return to Terminal and press Control-C to stop Streamlit.
 172. Keep only your exported evidence files; do not overwrite the supplied CSV.
 173. Run deactivate when you have finished the lab session.